

AiSD Lista 1

Dominik Kowalczyk

7 listopada 2024

1 Algorytm sortowania INSERTION SORT (przez wstawianie)

W podstawowej wersji (bez modyfikacji) algorytmu sortowania przez wstawienie wyróżniamy następujący fragment kodu:

Listing 1: Kod dla funkcji sortującej przez wstawienie

```
1 void INSERTION_SORT(int A[], int n) {  
2     for (int i = 1; i < n; i++) {  
3         int key = A[i];  
4         int j = i - 1;  
5         while (j >= 0 && A[j] > key) {  
6             A[j + 1] = A[j];  
7             j--;  
8         }  
9         A[j + 1] = key;  
10    }  
11 }
```

Powyższy fragment kodu zaczyna się od funkcji `for`, która przechodzi kolejno po każdym elemencie tablicy `A`. Naszą iterację zaczynamy od drugiego elementu tablicy (z indeksem 1) (zakładając, że 0 jest już posortowany). Zmienna `key` umożliwia przechowanie wartości wstawianego elementu zapobiegając nadpisaniu oraz pozwalając na porównywanie tej wartości z kolejnymi elementami posortowanego ciągu. Gdy napotkany element w posortowanej części jest większy od wstawianego, przesuwamy go o jedną pozycję w prawo, aby zrobić miejsce na wstawienie nowego elementu. Jeśli element w posortowanej części jest mniejszy lub równy wstawianemu, wówczas zapisujemy wstawiany element na kolejnej pozycji.

ilość elementów	10	100	1000	10000	50000
przypisanie	73	5877	506579	50242971	1248166021
porównanie	32	2889	252790	25116486	624058011

Rysunek 1: ilość przypisań oraz porównań dla INSERTION SORT

Modyfikacja INSERTION SORTA polega na wstawianiu "na raz" dwóch kolejnych elementów tablicy. Zatem duża część kodu działa w dość analogiczny sposób.

Listing 2: Fragment 1 funkcji sortującej przez wstawienie z modyfikacją

```

1 void INSERTION_SORT_MOD(int A[], int n) {
2     for (int s = 1; s < n - 1; s += 2) {
3         int pierwszy = A[s];
4         int drugi = A[s + 1];
5         if (pierwszy > drugi) {
6             swap(pierwszy, drugi);
7         }
8         int k = s - 1;
9         while (k >= 0 && A[k] > pierwszy) {
10             A[k + 1] = A[k];
11             k--;
12         }
13         A[k + 1] = pierwszy;
14         A[k + 2] = drugi;
15     }
16 }

```

Pętla zaczyna się od indeksu 1 i iteruje z krokiem +2, co pozwala jednocześnie rozpatrywać dwie wartości – *pierwszy* i *drugi*. Następnie porównujemy je za pomocą funkcji if i jeśli *pierwszy* jest większy zamieniamy miejscami. Funkcja while działa już na tej samej zasadzie co w kodzie bez modyfikacji.

Listing 3: Fragment 2 funkcji sortującej przez wstawienie z modyfikacją

```

1  if (n % 2 == 0) {
2      int ostatni = A[n - 1];
3      int k = n - 2;
4      while (k >= 0 && A[k] > ostatni) {
5          A[k + 1] = A[k];
6          k--;
7      }
8      A[k + 1] = ostatni;
9  }

```

Przedstawiona modyfikacja jednak wymaga rozważenia przypadku, czy tablica A ma długość parzystą, czy nieparzystą. Problem pojawia się gdy n jest parzyste, gdyż ostatni sortowany element nie ma pary ($ostatni = A[n - 1]$). Dla k przypisujemy przedostatni indeks tablicy, które też będzie miejscem rozpoczęcia wstawiania elementu $ostatni$. Pętla *while* działa identycznie jak ta w podstawowym INSERTION SORT, z uwagą, że naszym *key* jest *ostatni*.

2 Algorytm sortowania MERGE SORT (przez scalenie)

Listing 4: Fragment 1 funkcji sortującej przez scalenie bez modyfikacji

```

1  void MERGE(int A[], int p, int s, int k) {
2      int n1 = s - p + 1;
3      int n2 = k - s;
4      int L[n1 + 1];
5      int P[n2 + 1];
6      L[n1] = INT_MAX;
7      P[n2] = INT_MAX;
8  }

```

W przypadku MERGE SORTA ważną rolę odgrywa funkcja MERGE, na której na dobrą sprawę zbudowany jest MERGE SORT. MERGE przyjmuje cztery argumenty: p - początek, s - środek oraz k - koniec. Za ich pomocą dzielimy tablicę na dwie części: lewą ($n1 = s - p + 1$) oraz prawą ($n2 = k - s$). Definiujemy funkcje pomocnicze L i P o pojemności kolejno $n1 + 1$ oraz $n2 + 1$. Następnie

ustawiamy ostatni element na "napewno większy" od największego elementu tablicy $A[]$.

Listing 5: Fragment 2 funkcji sortującej przez scalenie bez modyfikacji

```
1 int i = 0;
2 int j = 0;
3 for (int l = p; l <= k; l++) {
4     if (L[i] <= P[j]) {
5         A[l] = L[i];
6         i++;
7     } else {
8         A[l] = P[j];
9         j++;
10    }
11 }
```

Po przypisaniu z lewej części do zmiennej L i z prawej części P resetujemy wskaźniki i oraz j (które wskazują naszą pozycję w kolejno w L i P).

Instrukcja $if(L[i] \leq P[j])$ sprawdza, czy bieżący element z L jest mniejszy lub równy bieżącemu elementowi z P .

Jeśli tak, to mniejszy element $L[i]$ jest wstawiany do tablicy $A[l]$, a wskaźnik i jest zwiększany o 1, by przejść do następnego elementu w L .

Jeśli nie, to mniejszy element $P[j]$ jest kopiowany do $A[l]$, a wskaźnik j jest zwiększany o 1, by przejść do następnego elementu w P .

Listing 6: Fragment 3 funkcji sortującej przez scalenie bez modyfikacji

```
1 void MERGESORT(int A[], int p, int k) {
2     if (p < k) {
3         int s = (p + k) / 2;
4         MERGESORT(A, p, s);
5         MERGESORT(A, s + 1, k);
6         MERGE(A, p, s, k);
7     }
8 }
```

Po przeanalizowaniu funkcji `MERGE`, możemy przejść do funkcji `MERGE SORT`, która wylicza srodek tablicy i wywołuje kolejno sortowanie z lewej strony, prawej strony po czym scala obie strony.

Listing 7: Fragment 1 funkcji sortującej przez scalenie z modyfikacją

```

1 void MERGEMODYFIKOWANY(int A[], int p, int s1, int s2, int k) {
2     int n1 = s1 - p + 1;
3     int n2 = s2 - s1;
4     int n3 = k - s2;
5     int L[n1 + 1];
6     int S[n2 + 1];
7     int P[n3 + 1];
8     L[n1] = INT_MAX;
9     S[n2] = INT_MAX;
10    P[n3] = INT_MAX;
11 }

```

Modyfikacja MERGE SORTA polegała na podzieleniu tablicy na trzy części zamiast dwóch, więc cały kod opiera się i ma bardzo podobną budowę do podstawowego MERGE SORTA. Zamiast dwóch części tworzymy trzy $n1 = s1 - p + 1$, $n2 = s2 - s1$, $n3 = k - s2$, a jako, że rozdzielamy tablicę. Pozostała reszta MERGA MODYFIKOWANEGO jak i MERGE SORTA MODYFIKOWANEGO przebiega bardzo podobnie.